JavaScript

시간복잡도와 공간복잡도

코드를 읽고 O(1), O(n), O(n^2)을 직접 계산하는 개념노트

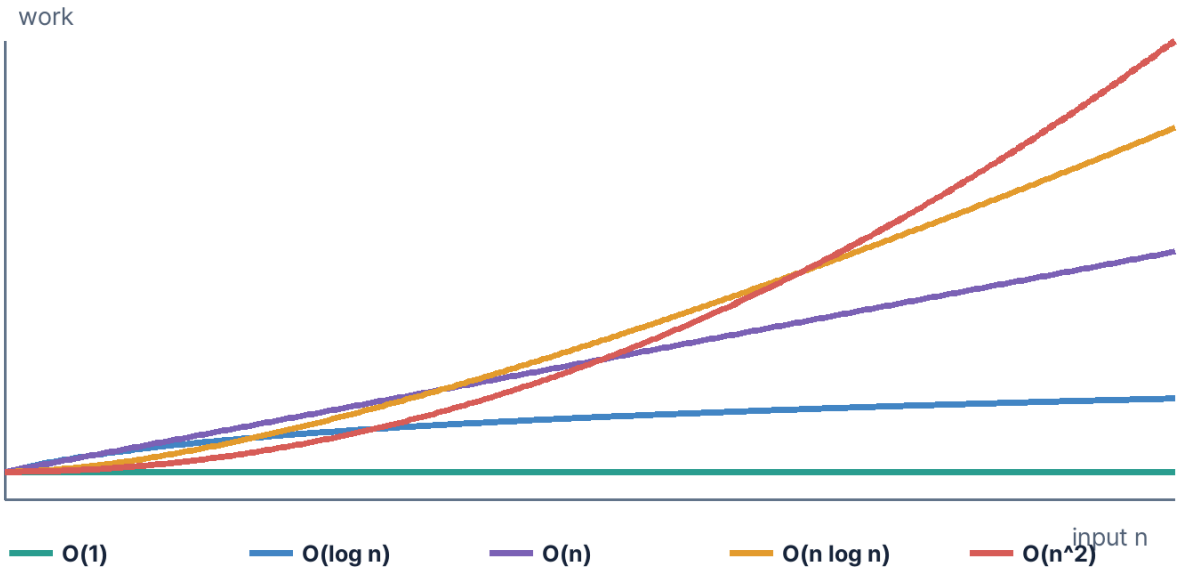


그림 1. 입력이 커질수록 작업량이 증가하는 모습. 곡선은 성장 형태를 보여 주며 실제 실행 시간을 뜻하지 않는다.

먼저 기억할 두 문장

시간복잡도: 입력 크기가 커질 때 작업 횟수가 늘어나는 방식.

공간복잡도: 입력 크기가 커질 때 필요한 메모리가 늘어나는 방식.

복잡도는 초 단위 실행 시간이나 바이트 수를 정확히 예측하지 않는다. 입력 증가에 따른 성장률을 비교한다.

학습 순서 ① 입력 정의 → ② 작업 횟수 → ③ 추가 메모리 → ④ JavaScript 연산 → ⑤ 직접 분석

1. 입력 크기부터 정의하기

n은 보통 배열 길이. 입력이 둘이면 n, m을 따로 둔다.

```
function sum(numbers) {
  let total = 0;
  for (const number of numbers) total += number;
  return total;
}
```

n = numbers.length. 반복문은 n번 실행한다. 변수 수는 입력 길이와 무관하다.

2. Big O 표기 읽기

표기	뜻	전형적 예
O(1)	일정	arr[i]
O(log n)	절반씩 감소	정렬 배열의 이진 탐색
O(n)	입력에 비례	전체 순회
O(n log n)	선형 × 로그	효율적 비교 정렬
$O(n^2)$	제곱으로 증가	모든 원소 쌍 비교
$O(2^n)$	지수로 증가	일부 완전 탐색

보통 최악의 경우를 분석하지만, 최선·평균·최악 중 무엇인지 명시해야 한다. O(1)이 작은 입력에서 무조건 가장 빠르다는 뜻은 아니다.

3. 시간복잡도 계산 규칙

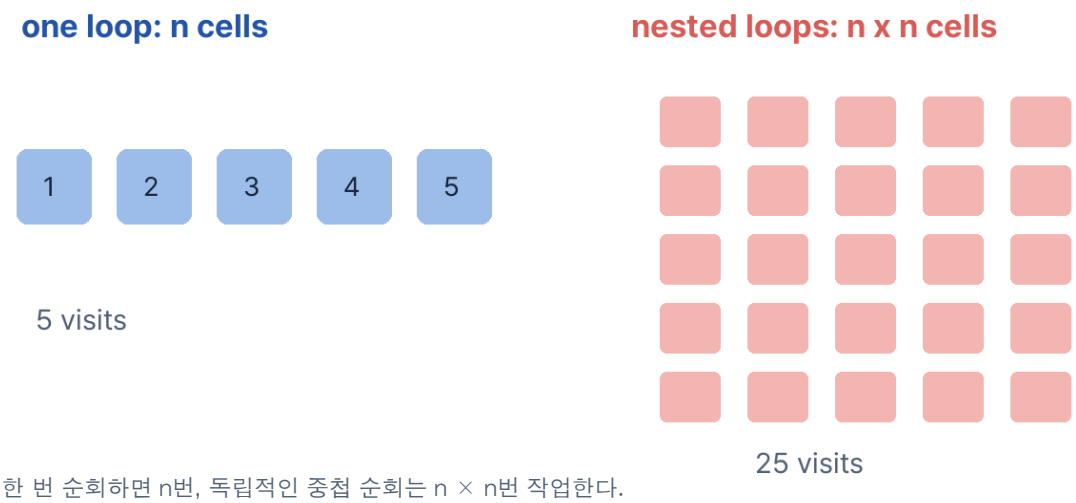


그림 2. 한 번 순회하면 n번, 독립적인 중첩 순회는 n × n번 작업한다.

코드 구조	계산	결과
한 번 실행	횟수 일정	O(1)
한 번 순회	n번	O(n)
순차 순회	n + m번	O(n + m)
중첩 순회	n × m번	O(nm)
매번 절반	약 $\log_2 n$번	O(log n)

단순화: $O(3n^2 + 10n + 7) \rightarrow O(n^2)$. 상수와 낮은 차수를 생략한다. 단, 서로 독립적인 입력 n과 m을 근거 없이 하나로 합치지 않는다.

들여쓰기만 세지 말 것: 투 포인터에서 안쪽 포인터가 실행 전체에 걸쳐 최대 n번만 움직이면 중첩처럼 보여도 O(n)이다.

4. 최선·평균·최악과 이진 탐색

```
function findIndex(a, target) {
  for (let i = 0; i < a.length; i++) {
    if (a[i] === target) return i;
  }
  return -1;
}
```

첫 원소에서 발견: 최선 O(1). 위치가 고르게 분포한다는 가정 아래: 평균 O(n). 마지막에 발견하거나 없음: 최악 O(n).

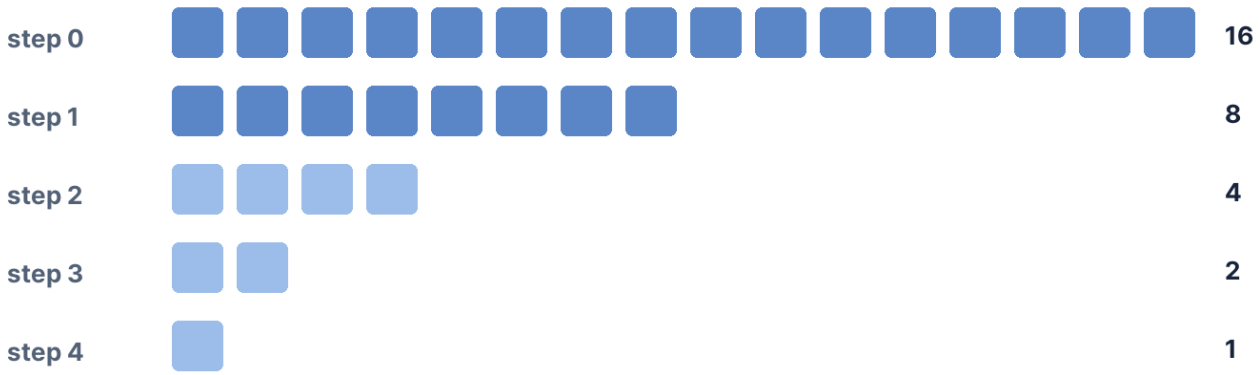


그림 3. 탐색 범위를 16 → 8 → 4 → 2 → 1로 줄이면 O(log n). 이진 탐색은 정렬된 배열이 필요하다.

5. 공간복잡도와 재귀

원래 입력을 제외하고 쓰는 메모리를 보조 공간이라 한다. 반환용 새 배열은 보통 추가 공간에 포함한다.

```
function doubled(a) {
  const out = [];
  for (const x of a) out.push(x * 2);
  return out;
}
```

시간 O(n), 추가 공간 O(n). 반면 합계 변수 하나만 쓰는 sum 함수는 추가 공간 O(1). 재귀는 호출 스택도 계산한다: factorial(n)은 시간 O(n), 스택 공간 O(n).

6. 시간을 줄이고 메모리를 더 쓰기

```
function hasDuplicate(a) {
  const seen = new Set();
  for (const x of a) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}
```

Set 조회·삽입을 평균 O(1)로 보면 전체 평균 시간 O(n), 공간 O(n). 모든 쌍을 직접 비교하면 최악 시간 $O(n^2)$, 추가 공간 O(1).

7. JavaScript에서 자주 만나는 연산

연산	일반적인 시간	추가 공간 / 주의
arr[i]	O(1)	인덱스 접근
push / pop	평균 O(1)	배열 끝
shift / unshift	O(n)	앞쪽 원소 이동 가능
includes / indexOf	O(n)	순차 탐색
slice / [...arr]	O(n)	복사본 O(n)
map / filter	O(n) + 콜백	결과 배열 보통 O(n)
sort(compareFn)	보통 O(n log n)	엔진·입력에 영향
Set.has / Map.get	평균 O(1)	해시 기반 조회

표의 값은 일반적인 구현 기준. JavaScript 명세가 이 연산들의 특정 Big O를 모두 보장하지는 않는다.

8. 숨은 비용 찾아내기

```
for (let i = 0; i < n; i++) {
  numbers.includes(i); // each call: O(n)
}
```

반복문 하나지만 내부의 includes가 매번 O(n) → 전체 $O(n^2)$. 함수 호출 비용까지 계산한다. slice, spread, map, filter는 새 배열 생성도 확인한다.

문자열은 변경할 수 없다. 반복적인 `text += part`의 실제 복사 비용은 엔진 최적화에 좌우된다. 성능 문제가 확인되면 조각을 모아 `join('')`을 검토한다.

9. 분석 순서: 이 5가지만 확인

① n, m 정의 → ② 각 연산과 호출 함수 비용 확인 → ③ 순차 실행은 더하고 독립적인 중첩은 곱하기 → ④ 최선·평균·최악 구분 → ⑤ 배열·Set·Map·재귀 스택 계산.

```
function matches(users, names) {
  const blocked = new Set(names);
  const result = [];
  for (const user of users) {
    if (blocked.has(user.name)) result.push(user);
  }
  return result;
}
```

n = 사용자 수, m = 이름 수. Set 생성 평균 O(m), 사용자 순회 평균 O(n), 반환 배열 최대 O(n). 따라서 평균 시간 O(n + m), 추가 공간 O(n + m).

최종 암기 카드

질문	답
작업을 몇 번 하나?	시간복잡도
새로 무엇을 저장하나?	공간복잡도
반복문 안에서 무엇을 호출하나?	숨은 비용 확인
재귀를 쓰나?	호출 스택 확인